

Technical Evidence and Claim Boundaries

Source-role map, boundaries and reviewer proof

Sanitized source map

This appendix maps the evidence by role and boundary without exposing private file paths or confidential access details.

Layer	Evidence contribution	Boundary
Remote WhatsApp-to-Agent Command Loop	Workflow notes describing WhatsApp event detection, audio transcription, structured work orders, model hints, and quality-loop thresholds.; Pipeline notes mapping WhatsApp -> OpenClaw gateway -> watcher -> MiguelAgent orchestration -> execution/evaluation/formatting -> WhatsApp reply.	Prototype/evolved system. Some EXOVIUM pieces were later superseded by Miguel Core. Private channel identifiers and credentials are excluded.
Miguel Core Unified Daemon	Miguel Core daemon source documenting the consolidation of earlier daemon/command surfaces into one local control plane.; Route map covering chat, streaming, voice, state, vault, knowledge, prediction, cascade, agents, orchestra, macOS/tool control, conversations, and health surfaces.	Local experimental control plane, not a hardened commercial security product.
Multi-Model Cascade Router	AI router source with provider registry for Gemini, Groq, Cerebras, OpenRouter, DeepSeek, Together, Ollama/local models, and LiteLLM-backed calls.; Cascade source implementing local/free/paid tiers, quota and cooldown tracking, quality gates, free-model racing, cancellation, paid-budget blocking, and outcome logging.	Cost-reduction claims should be treated as design goals unless fresh logs verify exact percentages.
Memory, Vault, CASS, and Smart Memory MCP	Memory Broker source showing broadcast schema, write logs, weight/centrality scoring, and lightweight contradiction detection.; CASS source implementing episodic raw memory, working summaries, procedural rules, confidence, decay, and SQLite persistence.	Presented as practical memory infrastructure, not proof of human-like consciousness.

Layer	Evidence contribution	Boundary
Voice Interface and Spoken Tool Use	Voice source implementing microphone streaming, Gemini Live configuration, tool declarations, dispatcher, vault tools, deep-think route, constrained shell, reconnect, and fallback behavior.; Voice pipeline notes defining wake-word, local STT, hybrid TTS including local/provider voices, barge-in, latency budget, and privacy red lines.	Wake-word/macOS app features should be described as planned or partially integrated unless retested.
ANIMA Experimental Kernel	ANIMA Python package with kernel, state, primitives, temporal modules, metrics, bridge/shell modules, tests, and benchmarks.; Evidence-status and methodology files that bound claims and identify current benchmark limitations.	Never present as proof of artificial consciousness. Use as an ambitious, testable experimental architecture.

Reusable architecture pattern

Ingress is separated from interpretation, execution, model choice, memory, evaluation, audit and reply.

- Remote messages and voice commands are authority surfaces.
- Model routing is treated as governance.
- Memory is treated as provenance and reliability infrastructure.

Technical reading map: Remote WhatsApp-to-Agent Command Loop

What I had in mind was simple but technically demanding: I wanted to be away from my laptop and still delegate meaningful work to it without turning my phone into a blind remote shell. The interesting part was the trust boundary. A message had to become a work order, the work order had to be executed by the right side of the system, and the result had to survive evaluation before coming back to me.

- WhatsApp message or audio arrives through OpenClaw.
- A watcher normalizes inbound events, handles media, deduplicates repeated messages, and dispatches work.
- MiguelAgent interprets the request into goal, complexity, work order, success criteria, and model hint.
- The execution side uses EXOVIUM/Miguel tooling and model routing.
- A quality loop evaluates the result and can ask for an improved work order before replying.
- The final response is formatted and sent back through WhatsApp.
- Workflow notes describing WhatsApp event detection, audio transcription, structured work orders, model hints, and quality-loop thresholds.
- Pipeline notes mapping WhatsApp -> OpenClaw gateway -> watcher -> MiguelAgent orchestration -> execution/evaluation/formatting -> WhatsApp reply.
- Python watcher and MiguelAgent modules showing event parsing, media handling, deduplication, work-order execution, evaluator loop, and response formatting.
- Authority boundary thinking: the remote channel is convenient, but the system treats it as a risky control surface.
- Operational thinking: immediate acknowledgement, deduplication, media handling and final reporting are all separate failure points.
- Evaluator-loop intuition: the system does not only run a task; it checks whether the result satisfies the original work order before returning it.

Technical reading map: Miguel Core Unified Daemon

Miguel Core came from a very practical frustration: I had too many strong pieces living as separate scripts and services. I wanted one local control layer that could make the system observable, routable and easier to reason about.

- One daemon replaces fragmented service surfaces.
- Routes cover chat/streaming, voice, state, vault, knowledge, prediction, cascade, agents, metacognition, orchestra, macOS/tool endpoints, and conversations.
- Imports connect memory broker, CASS pipeline, vault watcher, prediction, metacognition, state graph, orchestra, and macOS bridge.
- Startup/handover scripts define model routing, state probes, boot checks, and autonomy rings.
- Miguel Core daemon source documenting the consolidation of earlier daemon/command surfaces into one local control plane.
- Route map covering chat, streaming, voice, state, vault, knowledge, prediction, cascade, agents, orchestra, macOS/tool control, conversations, and health surfaces.
- Orchestra worker module implementing delegated async agent tasks, timeouts, cancellation, status, and result capture.
- Architecture consolidation: recognizing fragmentation and pulling many surfaces into one controllable layer.
- Operational maturity: boot probes and handover docs show that the system was meant to be run, not only admired.
- Boundary maturity: autonomy rings separate what the system may do from what must remain user-authorized.

Technical reading map: Multi-Model Cascade Router

I did not want to treat model choice as taste. I wanted routing to be part of the architecture: cheap when possible, strong when necessary, local when privacy matters, and fallback-aware when a provider fails.

- Task strategy selects speed, quality, free, code, creative, bulk, or local routing.
- Local models are tried before free APIs and paid tiers where appropriate.
- Quota/cooldown tracking blocks bad or unavailable routes.
- Quality gates decide whether to escalate.
- Free API models can be raced in parallel and slower calls cancelled after a sufficient answer.
- Outcomes are logged for later learning.
- AI router source with provider registry for Gemini, Groq, Cerebras, OpenRouter, DeepSeek, Together, Ollama/local models, and LiteLLM-backed calls.
- Cascade source implementing local/free/paid tiers, quota and cooldown tracking, quality gates, free-model racing, cancellation, paid-budget blocking, and outcome logging.
- LiteLLM configuration separating local T0, free API T1, and paid T2 tiers.
- Provider-neutral thinking: no single model is treated as magic.
- Governance thinking: cost, quality, privacy and quota are part of model selection.
- Measurement path: outcome logging creates a basis for later empirical routing decisions.

Technical reading map: Memory, Vault, CASS, and Smart Memory MCP

The memory work came from noticing that agents do not only need more context. They need memory hygiene: raw facts, summaries, procedural learning, contradictions, staleness and provenance.

- CASS keeps episodic raw records, structured working memory, and procedural rules with confidence/decay.
- Memory Broker broadcasts high-value or central facts across memory systems and flags contradictions.
- Vault watcher design handles file-change detection, incremental reindexing, graph updates, staleness propagation, and forgetting scores.
- Smart Memory MCP provides a simple local protocol component for learn/recall/stats/patterns/suggestions.
- Memory Broker source showing broadcast schema, write logs, weight/centrality scoring, and lightweight contradiction detection.
- CASS source implementing episodic raw memory, working summaries, procedural rules, confidence, decay, and SQLite persistence.
- Vault watcher design notes covering file-change detection, incremental indexing, graph updates, staleness propagation, and forgetting score.
- Context integrity: summaries are useful but dangerous when they replace raw evidence.
- Contradiction handling: conflicting facts are not simply overwritten.
- Agent safety relevance: stale memory can cause wrong action, wrong confidence and wrong long-term behavior.

Technical reading map: Voice Interface and Spoken Tool Use

Voice was exciting because it made the system feel immediate, but that is also why it is dangerous. If I can speak and the computer acts, then muting, confirmation, shell limits and privacy are part of the product, not accessories.

- Python terminal voice interface streams microphone audio into Gemini Live.
- The Live config exposes tools for vault search/read/write, deep thinking through Miguel Core, brain status, context, and constrained shell execution.
- Dangerous shell patterns are blocked and command execution is timeout-limited.
- A later design adds wake-word, local STT, hybrid TTS, barge-in, and privacy boundaries.
- Voice source implementing microphone streaming, Gemini Live configuration, tool declarations, dispatcher, vault tools, deep-think route, constrained shell, reconnect, and fallback behavior.
- Voice pipeline notes defining wake-word, local STT, hybrid TTS including local/provider voices, barge-in, latency budget, and privacy red lines.
- VoiceBridge notes separating browser/orb STT work from the local daemon response path.
- Human-interface realism: voice lowers friction so much that authority checks must become stronger.
- Privacy realism: always-listening systems should prefer local handling and explicit boundaries.
- Safety realism: barge-in is a control feature, not a luxury.

Technical reading map: ANIMA Experimental Kernel

ANIMA was my way of taking a very ambitious idea and forcing it into code, tests and evidence. The point is not to claim consciousness. The point is to show that even a fuzzy idea can be disciplined into architecture, metrics and boundaries.

- Kernel/state/type modules.
- Primitives such as qualia, engram, valence, nexus, impulse, trace, mirror, and flux.

- Temporal substrate and metric layers.
- Benchmarks and ablation methodology.
- Evidence status file that overrides over-strong claims.
- ANIMA Python package with kernel, state, primitives, temporal modules, metrics, bridge/shell modules, tests, and benchmarks.
- Evidence-status and methodology files that bound claims and identify current benchmark limitations.
- Local rebuild test run with 446 passing tests and one configuration warning.
- Ambition with restraint: the project reaches high but avoids claiming artificial consciousness as proven.
- Measurement instinct: benchmarks and ablations exist even when results are not decisive.
- Evidence hierarchy: results and methodology files outrank optimistic overview language.